

PyNvVideoCodec

Read Me

Table of Contents

Chapter 1. Release Notes 1
Chapter 2. System Requirements 4
Chapter 3. Installing PyNvVideoCodec Python Module 6
Chapter 4. Running Sample Applications 8
4.1. Basic Sample Applications 10
4.2. Jupyter Notebooks 12
4.3. Advanced Sample Applications 13

Chapter 1. Release Notes

Key Features and Enhancements

- **Decode statistics extraction:** Enables extraction of low-level decode statistics—such as QP values, coding-unit types, and motion vectors—from H.264 and H.265 streams. These statistics provide valuable insights into video quality, content complexity, and encoder behaviour.

> **Note:** The decode statistics feature requires NVIDIA display driver version 590 or newer.

- **Jupyter notebooks:** Added two step-by-step, interactive Jupyter notebook tutorials—one demonstrating the SimpleDecoder API for easy video decoding and flexible frame sampling , and another showcasing the use of ThreadedDecoder in a real-world deep-learning object detection workflow.
- **Enhanced sample applications:** Simplified, restructured and enhanced sample applications along with documentation to help developers learn, experiment, and build faster.
- **Enhanced Documentation:** Comprehensive API reference and programming guide, enriched with practical code snippets to help developers quickly understand and use the library.

Deprecation Notices

The following features and methods are deprecated in this release and will be removed in future versions:

- **nvcv_image() method:** The `nvcv_image()` method in the `DecodedFrame` class is deprecated and will be removed in a future version. This method was originally designed as a workaround for CV-CUDA tensor representation but is no longer the recommended approach.

Users are encouraged to migrate to alternative methods for CV-CUDA tensor conversion. The deprecated method will continue to function in this release but will emit deprecation warnings.

> **Important:** Deprecated features may be removed without further notice in major version updates. Please update your code to use supported alternatives.

Limitations and Known Issues

- PyNvVideoCodec uses the FFmpeg binaries for demuxing and muxing of audio and video content.

  NVIDIA will not update the FFmpeg binaries included in our release package as these binaries are available, maintained and updated by the FFmpeg open-source community.

  > **ATTENTION:** NVIDIA does not provide support for FFMPEG; therefore, it is the responsibility of end users and developers, to stay informed about any vulnerabilities or quality bugs reported against FFMPEG. Users are encouraged to refer to the official FFmpeg website and community forums for the latest updates, patches, and support related to FFmpeg binaries and act as they deem necessary.

- **WebM Container Seeking Limitations:** WebM containers may experience reduced seek accuracy due to codec-specific behavior of VP8/VP9 streams.

  During decode, some frames in VP8/VP9 (commonly used in WebM containers) are marked as non-displayable, causing discrepancies between the reported total frame count from container metadata and the actual displayable frame count. This can result in frame count mismatches and potential seeking issues near the end of video streams.

  PyNvVideoCodec implements workarounds to handle these discrepancies, including special packet filtering and frame count adjustments for WebM containers. However, users should be aware that seek operations may be less precise compared to other container formats like MP4.

  > **Note:** Similar limitations may also affect FLV container that use VP8/VP9 codecs.

Package Contents

This package contains the following:

1. Sample applications demonstrating usage of PyNvVideoCodec APIs for encoding, decoding and transcoding use cases.
   - [.\samples\basic] - Basic sample applications
   - [.\samples\advanced] - Advanced sample applications
2. Jupyter notebooks demonstrating usage of PyNvVideoCodec APIs.
   - [.\samples\jupyter]
3. Requirements files specifying dependencies.
   - [.\samples\requirements.txt] - Required libraries to run the sample applications and Jupyter notebooks
   - [.\benchmarks\requirements.txt] - Required libraries to run the benchmark scripts

4. Python Bindings
   - [.src\PyNvVideoCodec]
5. Video codec helper classes and utilities
   - [.src\VideoCodecSDKUtils]
6. FFmpeg libraries and source code
   - [.external\ffmpeg]
7. Documents
   - [.docs]
8. Benchmarks contains performance benchmarking scripts for testing various PyNvVideoCodec features including segmented transcoding, decoder caching, and frame sampling capabilities.
   - [.\benchmarks\]

The sample applications provided in the package are for demonstration purposes only and may not be fully tuned for quality and performance. Hence the users are advised to do their independent evaluation for quality and/or performance.

Chapter 2. System Requirements

Windows, Linux, and Windows Subsystem for Linux (WSL)

PyNvVideoCodec is supported on Windows, Linux, and Windows Subsystem for Linux (WSL). The following requirements apply to all supported platforms:

Table 1. System Requirements

Operating System	▶ Windows 10 or higher ▶ Ubuntu 18.04 or higher
GPU	▶ Turing ▶ Ampere ▶ Ada ▶ Hopper ▶ Blackwell
Drivers	**Pre-Blackwell GPUs:** ▶ NVIDIA Windows display driver 531.61 or newer ▶ NVIDIA Linux display driver 530.41.03 or newer **Blackwell GPUs and onwards:** ▶ NVIDIA Windows display driver 576.52 or newer ▶ NVIDIA Linux display driver 570.153.02 or newer Get most recent NVIDIA Display Driver
Python	▶ Python 3.10

	▶ Python 3.10 Dev (required in Ubuntu only)
CMake	▶ 3.21 and onwards
Visual Studio(Windows only)	▶ Visual Studio
CUDA Toolkit	Latest CUDA Toolkit
Python modules to run Sample applications	PyCUDA and PyTorch

Additional Configuration for WSL

In addition to all the requirements listed above, WSL users need the following configuration:

- Add the directory `/usr/lib/wsl/lib` to the PATH environment variable if it is not added by default. This is required to ensure the WSL libraries are included in the system path.

Chapter 3. Installing PyNvVideoCodec Python Module

ATTENTION: PyNvVideoCodec will download and install additional third-party open source software projects - DLPack. Review the license terms of these open source projects before use.

The Python module can be installed using following ways.

Installing from PyPI

1. The ready-to-use Python WHL's (Wheel) of the PyNvVideoCodec for Windows, Linux and WSL (Windows Subsystem for Linux) are hosted on PyPI.
2. Open the bash/shell prompt and run:

```
$>pip install PyNvVideoCodec
```

This is the recommended way to install PyNvVideoCodec.

Upon installation of the wheel, the sample applications and benchmark scripts are placed in the Python site-packages directory. The specific location of site-packages may vary depending on the operating system and Python environment. The path can be identified by running:

```
import site; print(site.getsitepackages())
```

Building and Installing from Source on NVIDIA NGC

The package containing PyNvVideCodec Python module's source code, all dependencies, Python sample applications, and documents is hosted on NVIDIA NGC.

Follow these steps:

1. Download the zip file of the latest package from NVIDIA NGC .
2. Open the bash/shell prompt from the same directory where zip was downloaded and run the following command, replacing "PyNvVideoCodec.zip" with the actual name of the downloaded zip file:

```
$>pip install "PyNvVideoCodec.zip"
```

3. You can access documents and Python sample applications from the package.

Use this method if you need any customization on PyNvVideoCodec Python module e.g. enabling NVTX markers for profiling.

Follow these steps to build customized version:

1. Unzip the source package to a directory.
2. Do the necessary modifications to the source.
3. On the same directory where `setup.py` is located, run the following commands:

```
$>pip install .
```

Chapter 4. Running Sample Applications

PyNvVideoCodec includes a comprehensive set of sample applications and Jupyter notebooks that demonstrate the API usage across various use cases. These samples are organized into three categories:

- **Basic Sample Applications** [.\samples\basic]: Demonstrate fundamental encoding, decoding, and transcoding workflows suitable for getting started with PyNvVideoCodec.
- **Advanced Sample Applications** [.\samples\advanced]: Showcase advanced features such as low-latency decoding, SEI message handling, performance measurement, and CUDA resource management.
- **Jupyter Notebooks** [.\samples\jupyter]: Provide interactive tutorials that guide users through the API with step-by-step explanations and live code execution.

Prerequisites

Ensure you have the following prerequisites before running the samples:

- NVIDIA GPU with appropriate drivers installed
- PyNvVideoCodec module is installed
- Required dependencies installed by running the following command on shell prompt:

```
pip install -r samples/requirements.txt
```

Sample Applications Overview

The following table provides an overview of all sample applications and their objectives:

Type	Application Name	Objective
Basic	simple_decode_sampling.py	Decode multiple video files and sample frames at regular intervals. Converts decoded frames to PyTorch tensors for use in machine learning workflows.
	encode.py	Encode raw video frames into a compressed video file. Supports reading input from both CPU memory and GPU memory.

Type	Application Name	Objective
	create_video_segments.py	Extract specific portions of a video based on start and end timestamps. Useful for creating training clips or trimming videos.
Jupyter	simple_decode_tutorial.ipynb	Step-by-step tutorial for video decoding. Learn how to access frames by index, retrieve video metadata, and reuse the decoder for multiple files.
	object_detection_tutorial.ipynb	Run object detection on video frames using a pre-trained deep learning model. Uses threaded decoding to run video decoding and AI inference in parallel.
Advanced	decode.py	Reads a video file using low level core decoder interface and outputs raw decoded frame
	decode_with_cuda_control.py	Decode video with explicit control over CUDA resources. Useful when integrating with other CUDA-based libraries that require specific context or stream management.
	decode_from_memory_buffer.py	Decode video data directly from memory instead of reading from a file. Useful for processing video received over a network or stored in memory.
	decode_with_low_latency.py	Decode video with minimal delay between input and output. Useful for real-time applications like video conferencing or live streaming.
	decode_perf.py	Measure video decoding speed using multiple decoder instances in parallel. Helps evaluate system performance and throughput.
	decode_sei_msg.py	Extract metadata embedded in the video stream (SEI messages). Used to retrieve information like HDR settings, timecodes, or custom data.
	simple_decode_stats.py	Extracts decode stats QP values, coding-unit types, and motion vectors from video stream
	decode_reconfigure.py	Dynamically reconfigure the core decoder to handle resolution changes. Switch between input streams with different dimensions without recreating the decoder.
	encode_reconfigure.py	Change encoding settings (like bitrate) while encoding is in progress. Useful for adaptive streaming where quality adjusts based on network conditions.
	encode_sei_msg.py	Embed custom metadata into the encoded video stream. Used to add HDR information, timecodes, or application-specific data.

Type	Application Name	Objective
	encode_perf.py	Measure video encoding perf using multiple encoder instances in parallel. Helps evaluate system performance and throughput.

4.1. Basic Sample Applications

The basic sample applications demonstrate fundamental video decode, encode and transcode workflows. These applications are ideal for getting started with PyNvVideoCodec.

simple_decode_sampling.py

This sample demonstrates multi-file video decoding with frame sampling and tensor conversion, which is a common use case in machine learning pipelines. The application accepts one or more video files as input and samples frames evenly across each video's duration. Using the SimpleDecoder API, frames are decoded directly to GPU memory in RGB format and converted to PyTorch tensors for downstream processing. The sample showcases decoder reconfiguration, allowing a single decoder instance to process multiple video files of different resolutions efficiently. Frames are sampled evenly across the video duration based on the specified frame count.

```
python simple_decode_sampling.py video1.mp4 video2.mp4 -g 0 -f 16
```

Parameter	Type	Description
video_files	string	One or more video files to process (positional arguments)
-g, --gpu-id	int	GPU device ID to use (default: 0)
-f, --frames	int	Number of frames to sample per video (default: 8)

encode.py

This sample demonstrates video encoding with support for both CPU and GPU buffer modes. The application reads raw YUV frames from an input file and encodes them using PyNvVideoCodec's encode API. In CPU buffer mode, frame data is read into host memory and copied to CUDA buffers for encoding. In GPU buffer mode, frame data is loaded directly into CUDA device buffers, reducing memory transfer overhead. The sample supports multiple codecs (H.264, HEVC, AV1) and various input formats (NV12, YUV444, P010, etc.). Encoder configuration can be customized via a JSON configuration file for fine-grained control over encoding parameters such as bitrate, GOP size, and quality presets.

```
python encode.py -i input.yuv -s 1920x1080 -m gpu -if NV12 -c h264
```

Parameter	Type	Description
-i, --input	string	Path to raw input video file (YUV format)
-o, --output	string	Path to output encoded video file

Parameter	Type	Description
-s, --size	string	Frame size in WxH format (e.g., 1920x1080)
-m, --mode	string	Buffer mode: cpu or gpu (default: gpu)
-if, --format	string	Input pixel format: NV12, YUV444, P010, ARGB, ABGR, etc. (default: NV12)
-c, --codec	string	Output codec: h264, hevc, av1 (default: h264)
-f, --frames	int	Number of frames to encode (default: all frames)
-g, --gpu-id	int	GPU device ID to use (default: 0)
-json	string	Path to JSON encoder configuration file

create_video_segments.py

This sample demonstrates creation of smaller video segments from bigger video file. This approach is useful for creating small meaning video clips for training of AI models. The application extracts segments from a video file based on timestamp ranges specified in a text file. The sample uses the SimpleDecoder to retrieve video metadata for duration validation, then uses the Transcoder API to extract each segment as a separate output file. Output files are automatically named with the timestamp range appended to the base filename. Timestamps that exceed the video duration are automatically clipped, and invalid ranges are reported with error messages.

```
python create_video_segments.py -i input.mp4 -s segments.txt -c transcode_config.json
```

Parameter	Type	Description
-i, --input	string	Path to input video file
-s, --segments	string	Path to segments file containing timestamp ranges (default: segments.txt)
-c, --config	string	Path to transcoder configuration JSON file (default: transcode_config.json)
-o, --output	string	Output filename template (timestamp is appended automatically)
-g, --gpu-id	int	GPU device ID to use (default: 0)

The segments file should contain one timestamp range per line in the format: `start_time end_time`

Example segments.txt:

```
0.0 10.5
15.0 30.0
45.5 60.0
```

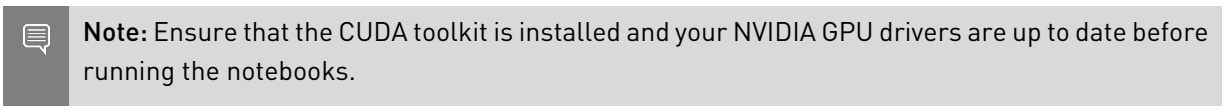
4.2. Jupyter Notebooks

The Jupyter notebooks provide interactive tutorials that guide users through the PyNvVideoCodec APIs with step-by-step explanations and live code execution. These notebooks are ideal for learning and experimentation.

Setting Up Jupyter Notebooks

To run the Jupyter notebooks, follow these steps:

1. Ensure PyNvVideoCodec is installed in your Python environment.
2. Install the required dependencies:
   ```
   pip install -r samples/requirements.txt
   ```
3. Navigate to the notebooks directory and launch Jupyter:
   ```
   cd samples/jupyter
   jupyter notebook
   ```
4. Open the desired notebook from the Jupyter interface in your browser.



Note: Ensure that the CUDA toolkit is installed and your NVIDIA GPU drivers are up to date before running the notebooks.

simple_decode_tutorial.ipynb

This notebook provides a comprehensive tutorial on using the SimpleDecoder API for GPU-accelerated video decoding. The tutorial demonstrates how to decode videos efficiently and access frames using various methods. The notebook covers multiple frame access methods: single frame access by index, seeking to specific frames, batch frame retrieval, accessing frames by specific indices, slice-based access, and time-based frame retrieval. Another key feature demonstrated is decoder reconfiguration, which allows reusing a single decoder instance to process multiple video files of different resolutions without the overhead of creating new decoder instances. This is particularly useful in deep learning pipelines where processing many short video clips efficiently is critical. The sample also displays decoded output frames.

Key Features Demonstrated:

- Basic decoder setup and initialization with SimpleDecoder
- Retrieving video stream metadata (resolution, frame count, FPS, duration)
- Advanced stream information including key frame locations and GOP structure
- Multiple frame access methods: index, batch, slice, and time-based
- Decoder reconfiguration for processing multiple videos efficiently
- Converting decoded frames to PyTorch tensors

object_detection_tutorial.ipynb

This notebook demonstrates a practical deep learning use case: real-time object detection in video using ThreadedDecoder is used to decode video in parallel. Unlike a normal decoder where decode and inference are executed sequentially (causing the inference to stall while waiting for frames), the ThreadedDecoder runs decoding on a separate background thread and keeps next batch of pre-decoded frames ready for inference. This ensures that decoded frames are always ready when the inference model needs them, eliminating pipeline stalls and maximizing GPU utilization. The tutorial downloads a sample video, sets up a ThreadedDecoder with configurable buffer size, and uses a pre-trained Faster R-CNN model with ResNet-50 backbone to detect objects in video frames. The model is trained on the COCO dataset and can identify 91 different object classes. Detection results are filtered by confidence threshold and displayed with bounding boxes and class labels overlaid on each frame. This notebook serves as a template for building efficient video analytics applications.

Key Features Demonstrated:

- Understanding the difference between normal decoder and ThreadedDecoder
- Setting up ThreadedDecoder with buffer size configuration
- Integrating video decoding with PyTorch deep learning models
- Running Faster R-CNN object detection on decoded frames
- Filtering detection results by confidence threshold
- Visualizing object detection results with bounding boxes and labels
- Building efficient parallel decode-inference pipelines

4.3. Advanced Sample Applications

This section demonstrates advanced features of PyNvVideoCodec including low-latency decoding, SEI message handling, performance measurement, CUDA resource management, and decode statistics extraction. These samples are designed for users who need fine-grained control over video processing workflows.

decode.py

This sample demonstrates basic video decoding using the core decoder API with a demuxer. The application creates a demuxer to extract encoded packets from a video file and sets up a core decoder for hardware-accelerated decoding.

The decode pipeline in this sample is as follows:

video file -> demuxer -> packets -> decoder -> raw YUV frames.

The sample shows how to work with both device and host memory for frame data, query hardware capabilities like the number of NVDEC engines, limit the number of frames decoded using the frame count parameter. Output is written as raw YUV frames to a file.

```
python decode.py -i input.mp4 -o output.yuv -d 1
```

Parameter	Type	Description
-i, --input	string	Path to input video file
-o, --output	string	Path to output raw YUV file (default: <input_name>.yuv)
-d	int	Use device memory (1) or host memory (0) (default: 1)
-g, --gpu-id	int	GPU device ID to use (default: 0)
-f, --frames	int	Number of frames to decode (default: all frames)

decode_with_cuda_control.py

This sample demonstrates advanced video decoding with explicit CUDA context and stream management. This application gives full control over CUDA resources by manually initializing CUDA contexts and streams before creating the decoder. This is essential for applications that need to share CUDA resources across multiple components or require precise control over GPU memory and synchronization. The sample shows proper CUDA resource cleanup order (decoder, demuxer, stream, context), switching between device memory (zero-copy) and host memory modes, and querying hardware capabilities.

```
python decode_with_cuda_control.py -i input.mp4 -o output.yuv -d 1
```

Parameter	Type	Description
-i, --input	string	Path to input video file
-o, --output	string	Path to output raw YUV file (default: <input_name>.yuv)
-d	int	Use device memory (1) or host memory (0) (default: 1)
-g, --gpu-id	int	GPU device ID to use (default: 0)
-f, --frames	int	Number of frames to decode (default: all frames)

decode_from_memory_buffer.py

This sample demonstrates decoding video directly from memory. This approach is useful when video data comes from network streams, memory-mapped files, or any source where data is available in memory rather than as a file on disk. The application implements a custom VideoStreamFeeder class that reads video data into memory and feeds chunks to the demuxer through a callback mechanism. This saves file I/O overhead and enables streaming applications. The sample shows how to create a callback-based demuxer, manage video data buffers and chunk sizes efficiently, implement proper buffer position tracking and EOF handling, and work with both device and host memory for decoded frames.

```
python decode_from_memory_buffer.py -i input.mp4 -o output.yuv -d 1
```

Parameter	Type	Description
-i, --input	string	Path to input video file
-o, --output	string	Path to output raw YUV file (default: <input_name>.yuv)
-d	int	Use device memory (1) or host memory (0) (default: 1)
-g, --gpu-id	int	GPU device ID to use (default: 0)
-f, --frames	int	Number of frames to decode (default: all frames)

decode_with_low_latency.py

This sample demonstrates low-latency video decoding using different latency modes. The decoder supports three latency modes:

- **NATIVE (0):** Has at least 1 frame latency for streams with B-frames and outputs in display order.
- **LOW (1):** Has zero latency for All-Intra and IPPP sequences (without B-frames) while maintaining display order.
- **ZERO (2):** Has zero latency for All-Intra/IPPP streams and outputs in decode order.

Low and zero latency modes should not be used with streams containing B-frames. For low latency modes, the sample sets the ENDOFPICTURE flag on packets to trigger immediate decode. This is useful for real-time video applications such as live streaming, video conferencing, and interactive media where minimizing decode latency is critical.

```
python decode_with_low_latency.py -i input.mp4 -o output.yuv -dl 1
```

Parameter	Type	Description
-i, --input	string	Path to input video file
-o, --output	string	Path to output raw YUV file (default: <input_name>.yuv)
-d	int	Use device memory (1) or host memory (0) (default: 1)
-g, --gpu-id	int	GPU device ID to use (default: 0)
-f, --frames	int	Number of frames to decode (default: all frames)
-dl	int	Decode latency mode: 0=NATIVE, 1=LOW, 2=ZERO

decode_perf.py

This sample demonstrates how to measure video decoding performance. The application supports two execution modes:

- **Thread mode:** Offers higher performance with lower overhead, shared memory access, and shared CUDA context between decoder instances. Python GIL is not a bottleneck as decoder operations are GIL-free.
- **Process mode:** Provides complete isolation between decoder instances with independent GPU memory.

The sample reports detailed performance metrics including frames per second (FPS), total frames decoded, and wall time. This is a performance testing application that does not write output files.

```
python decode_perf.py -i input.mp4 -n 4 -m thread
```

Parameter	Type	Description
-i, --input	string	Path to input video file
-n	int	Number of parallel instances (default: 1)
-m, --mode	string	Execution mode: thread or process (default: thread)
-d	int	Use device memory (1) or host memory (0) (default: 1)
-g, --gpu-id	int	GPU device ID to use (default: 0)
-f, --frames	int	Number of frames to decode per instance (default: all frames)

decode_sei_msg.py

This sample demonstrates how to extract and parse Supplemental Enhancement Information (SEI) messages from video streams during decoding. SEI messages are additional data embedded in video streams that provide supplementary information such as HDR/display metadata (color volume, light levels, transfer characteristics), timecode data for frame timing and sequence information, and custom metadata for application-specific needs. Common use cases include HDR display configuration for video playback, frame-accurate editing in content creation, timing synchronization in broadcast, and embedding application-specific metadata. The sample outputs raw binary SEI messages to one file and pickled SEI type information to another file, enabling further analysis or processing.

```
python decode_sei_msg.py -i input.mp4 -s sei_message.bin -st sei_type_message.bin
```

Parameter	Type	Description
-i, --input	string	Path to input video file
-s	string	Output SEI message file (default: sei_message.bin)
-st	string	Output SEI type message file (default: sei_type_message.bin)
-d	int	Use device memory (1) or host memory (0) (default: 1)
-g, --gpu-id	int	GPU device ID to use (default: 0)

simple_decode_stats.py

This sample demonstrates how to extract and analyze decode statistics from H.264 and H.265 video streams using the SimpleDecoder API. The statistics collected include:

- **QP (Quantization Parameter):** Analysis with average, min, and max values per frame indicating compression levels.

- **CU (Coding Unit) type distribution:** Shows INTRA (spatial prediction), INTER (temporal prediction), SKIP (copy from reference), and PCM (uncompressed) blocks.
- **Motion vector statistics:** For L0 and L1 references, enabling temporal complexity assessment.
- **Macroblock details:** Provides per-block encoding decisions and parameters.

These statistics are valuable for video quality analysis, encoder behavior understanding, performance optimization, and debugging encoding/decoding issues. The output is written as formatted text to a statistics file.

```
python simple_decode_stats.py -i input.mp4 -p output_stats.txt -d 1
```

Parameter	Type	Description
-i, --input	string	Path to input video file
-p	string	Output file for statistics (default: <input_name>_stats.txt)
-d	int	Use device memory (1) or host memory (0) (default: 1)
-g, --gpu-id	int	GPU device ID to use (default: 0)



Note: The decode statistics feature requires NVIDIA display driver version 590 or newer.

decode_reconfigure.py

This sample demonstrates dynamic core decoder reconfiguration for handling resolution changes during playback. The application shows how to switch between input streams with different dimensions without recreating the decoder, which is useful for adaptive streaming scenarios or processing multiple video files with varying resolutions. The sample creates a decoder with maximum dimensions to accommodate both streams, decodes frames from the first stream, uses `setReconfigParams()` to reconfigure for the second stream's dimensions, and continues decoding the second stream.

```
python decode_reconfigure.py -i1 video1.mp4 -i2 video2.mp4 -o1 output1.yuv -o2
 output2.yuv
```

Parameter	Type	Description
-i1	string	Path to first input video file
-i2	string	Path to second input video file (can have different resolution)
-o1	string	Path to first output raw YUV file (default: <input1_name>.yuv)
-o2	string	Path to second output raw YUV file (default: <input2_name>.yuv)
-d	int	Use device memory (1) or host memory (0) (default: 1)
-g, --gpu-id	int	GPU device ID to use (default: 0)

Parameter	Type	Description
-f, --frames	int	Number of frames to decode per stream (default: all frames)

encode_reconfigure.py

This sample demonstrates dynamic encoder reconfiguration for bitrate control at runtime. The application shows how to modify encoder parameters without resetting the encoder session, which is useful for adaptive bitrate streaming and dynamic quality adjustment. The sample changes the bitrate every 100 frames: at frame 0 it uses the original bitrate, at frame 100 it reduces to half the bitrate, at frame 200 it restores the original bitrate, and so on. The reconfiguration also handles VBV (Video Buffer Verifier) parameters including buffer size and initial delay. This capability is essential for live streaming applications that need to adapt to changing network conditions or for video conferencing systems that adjust quality based on bandwidth availability.

```
python encode_reconfigure.py -i input.yuv -s 1920x1080 -c h264
```

Parameter	Type	Description
-i, --input	string	Path to raw input video file (YUV format)
-o, --output	string	Path to output encoded video file
-s, --size	string	Frame size in WxH format (e.g., 1920x1080)
-if, --format	string	Input pixel format (default: NV12)
-c, --codec	string	Output codec: h264, hevc, av1
-f, --frames	int	Number of frames to encode (default: all frames)
-g, --gpu-id	int	GPU device ID to use (default: 0)
-json	string	Path to JSON encoder configuration file

encode_sei_msg.py

This sample demonstrates SEI (Supplemental Enhancement Information) message insertion during encoding. The application shows how to embed custom metadata into the encoded bitstream, which can be extracted during playback or transcoding. SEI messages support various types including HDR/display metadata (color volume, light levels, transfer characteristics), timecode data for frame timing, and custom user-defined data. The sample handles codec-specific SEI types: for H.264 and HEVC it uses SEI type 5 (user data unregistered), and for AV1 it uses type 6. Common use cases include embedding HDR metadata for proper display configuration, inserting timecodes for broadcast synchronization, and adding application-specific data for content management systems.

```
python encode_sei_msg.py -i input.yuv -s 1920x1080 -c hevc
```

Parameter	Type	Description
-i, --input	string	Path to raw input video file (YUV format)

Parameter	Type	Description
-o, --output	string	Path to output encoded video file (auto-generated as <input>.<codec>)
-s, --size	string	Frame size in WxH format (e.g., 1920x1080)
-if, --format	string	Input pixel format (default: NV12)
-c, --codec	string	Output codec: h264, hevc, av1
-f, --frames	int	Number of frames to encode (default: all frames)
-g, --gpu-id	int	GPU device ID to use (default: 0)
-json	string	Path to JSON encoder configuration file

encode_perf.py

This sample demonstrates advanced parallel video encoding using multiple threads or processes to achieve higher encoding throughput. Similar to decode_perf.py, it supports two execution modes:

- **Thread mode:** Runs multiple encoders in the same process with shared memory access, lower overhead, and simpler synchronization.
- **Process mode:** Runs multiple encoders in separate processes with IPC memory sharing, providing better isolation and stability.

The sample reports detailed performance metrics including total frames encoded, combined FPS across all workers, and average FPS per instance. This is a performance testing application that does not write output files.

```
python encode_perf.py -m thread -i input.yuv -s 1920x1080 -n 4
```

Parameter	Type	Description
-m, --mode	string	Execution mode: thread or process
-i, --input	string	Path to raw input video file (YUV format)
-s, --size	string	Frame size in WxH format (e.g., 1920x1080)
-if, --format	string	Input pixel format (default: NV12)
-c, --codec	string	Output codec: h264, hevc, av1
-n	int	Number of worker threads/processes (default: 1)
-f, --frames	int	Frames per worker (default: all frames)
-g, --gpu-id	int	GPU device ID to use (default: 0)
-json	string	Path to JSON encoder configuration file

Notice

This document is provided for information purposes only and shall not be regarded as a warranty of a certain functionality, condition, or quality of a product. NVIDIA Corporation ("NVIDIA") makes no representations or warranties, expressed or implied, as to the accuracy or completeness of the information contained in this document and assumes no responsibility for any errors contained herein. NVIDIA shall have no liability for the consequences or use of such information or for any infringement of patents or other rights of third parties that may result from its use. This document is not a commitment to develop, release, or deliver any Material (defined below), code, or functionality.

NVIDIA reserves the right to make corrections, modifications, enhancements, improvements, and any other changes to this document, at any time without notice.

Customer should obtain the latest relevant information before placing orders and should verify that such information is current and complete.

NVIDIA products are sold subject to the NVIDIA standard terms and conditions of sale supplied at the time of order acknowledgment, unless otherwise agreed in an individual sales agreement signed by authorized representatives of NVIDIA and customer ("Terms of Sale"). NVIDIA hereby expressly objects to applying any customer general terms and conditions with regards to the purchase of the NVIDIA product referenced in this document. No contractual obligations are formed either directly or indirectly by this document.

NVIDIA products are not designed, authorized, or warranted to be suitable for use in medical, military, aircraft, space, or life support equipment, nor in applications where failure or malfunction of the NVIDIA product can reasonably be expected to result in personal injury, death, or property or environmental damage. NVIDIA accepts no liability for inclusion and/or use of NVIDIA products in such equipment or applications and therefore such inclusion and/or use is at customer's own risk.

NVIDIA makes no representation or warranty that products based on this document will be suitable for any specified use. Testing of all parameters of each product is not necessarily performed by NVIDIA. It is customer's sole responsibility to evaluate and determine the applicability of any information contained in this document, ensure the product is suitable and fit for the application planned by customer, and perform the necessary testing for the application in order to avoid a default of the application or the product. Weaknesses in customer's product designs may affect the quality and reliability of the NVIDIA product and may result in additional or different conditions and/or requirements beyond those contained in this document. NVIDIA accepts no liability related to any default, damage, costs, or problem which may be based on or attributable to: (i) the use of the NVIDIA product in any manner that is contrary to this document or (ii) customer product designs.

Trademarks

NVIDIA, the NVIDIA logo, and cuBLAS, CUDA, CUDA Toolkit, cuDNN, DALI, DIGITS, DGX, DGX-1, DGX-2, DGX Station, DLProf, GPU, Jetson, Kepler, Maxwell, NCCL, Nsight Compute, Nsight Systems, NVCaffe, NVIDIA Deep Learning SDK, NVIDIA Developer Program, NVIDIA GPU Cloud, NVLink, NVSHMEM, PerfWorks, Pascal, SDK Manager, Tegra, TensorRT, TensorRT Inference Server, Tesla, TF-TRT, Triton Inference Server, Turing, and Volta are trademarks and/or registered trademarks of NVIDIA Corporation in the United States and other countries. Other company and product names may be trademarks of the respective companies with which they are associated.

Copyright

© 2010-2026 NVIDIA Corporation. All rights reserved.

NVIDIA Corporation | 2788 San Tomas Expressway, Santa Clara, CA 95051
http://www.nvidia.com